

Saksham Linkedin → <https://www.linkedin.com/in/sakshamarora9575/>

Topmate → <https://topmate.io/sakshamarora>

Setup Cursor → <https://topmate.io/sakshamarora/2040710>

Master Vibe coding → <https://topmate.io/sakshamarora/2079387>

## AI Tool

- <https://cursor.com/download>

## Optional

- Windsurf → <https://windsurf.com/download>
- Antigravity → <https://antigravity.google/>
- Qoder → <https://qoder.com/>

## STEPS

1. Create a docs folder (CONTEXT SETUP)
  - a. **problemStatement.md** → all the details of your problem statement
  - b. **architecture.md** → detailed architecture of the problem we are solving
  - c. **Implementation-plan.md** → phase-wise implementation
  - d. **edge-case.md** → corner scenario

## Prompt

1. Generate an detailed architecture.md file for this project using the @problemStatement.md
2. Generate a phase-wise implementation plan as per the @docs/architecture.md
3. Generate an edge-case.md file for each phase so that while coding or implementing this file can used to refer all the edge case @docs/implementation-plan.md

## Problem Statement

### Resume Shapeshifter — JD-to-Resume Tailoring Engine

#### 1. Project Summary

**Resume Shapeshifter** is a JD-to-resume tailoring engine. A user provides a job description and an existing resume, and the system generates a tailored version of the resume that better aligns with the job listing while preserving truthfulness and the user's actual experience.

The product should rewrite resume bullets, score the resume-to-JD match, flag missing skills or experience gaps, and generate a side-by-side PDF comparing the original resume with the tailored resume for a real job listing.

The goal is not to fabricate experience. The system should help users express their existing experience in language that better matches a target role.

---

## 2. Problem Statement

Job seekers often apply to many roles that require slightly different phrasing, skills, and emphasis. Tailoring a resume manually for each job description is time-consuming, inconsistent, and difficult to evaluate objectively.

Existing resume tools often provide generic suggestions, but they do not clearly show:

- How well the resume matches a specific job description.
- Which resume bullets should be rewritten.
- Which keywords, skills, or responsibilities are missing.
- Whether the tailored resume remains truthful to the original resume.
- A clean before-and-after comparison that can be reviewed and exported.

**Resume Shapeshifter solves this by ingesting a resume and job description, then producing a targeted resume rewrite with scoring, gap analysis, and a side-by-side proof artifact.**

---

## 3. Target Users

### Primary Users

- Job seekers applying to multiple roles.
- Students and early-career professionals tailoring resumes for internships or entry-level jobs.
- Mid-career professionals applying to role-specific openings.

### Secondary Users

- Career coaches.
  - Resume reviewers.
  - Bootcamp placement teams.
  - University career centers.
- 

## 4. Core Value Proposition

Given a resume and a job description, the product should answer:

1. **How well does this resume match the job?**
2. **What should be changed to improve the match?**
3. **Which bullets can be rewritten truthfully?**
4. **What gaps remain after tailoring?**
5. **What does the original vs tailored resume look like side by side?**

---

## 5. MVP Scope

The MVP should support the following workflow:

1. User uploads or pastes an existing resume.
2. User pastes a job description or URL text for a real job listing.
3. System parses both documents.
4. System extracts relevant skills, responsibilities, keywords, seniority signals, and role requirements from the JD.
5. System evaluates the current resume against the JD.
6. System rewrites resume bullets to better align with the JD.
7. System flags skills or requirements that are missing or weakly represented.
8. System generates a match score before and after tailoring.
9. System produces a side-by-side comparison: original resume vs tailored resume.
10. System exports the result as a PDF.

---

## 6. Non-Goals for MVP

The MVP should not attempt to:

- Apply to jobs automatically.
- Scrape job boards at scale.
- Fabricate work history, education, certifications, or metrics.
- Guarantee ATS ranking outcomes.
- Replace professional career advice.
- Support complex multi-column resume designs perfectly in version one.
- Create cover letters unless added later as an extension.

---

## 7. Key Product Requirements

### 7.1 Resume Input

Support at least one of the following in MVP:

- Plain text resume input.
- PDF resume upload.
- DOCX resume upload.

The parsed resume should preserve logical sections such as:

- Contact information.
- Summary.
- Skills.
- Work experience.
- Projects.
- Education.
- Certifications.

## 7.2 Job Description Input

Support pasted job description text in MVP.

Optional later enhancement:

- Accept a job posting URL and extract readable text.

## 7.3 JD Analysis

The system should extract:

- Job title.
- Company name, if present.
- Required skills.
- Preferred skills.
- Tools, technologies, and platforms.
- Responsibilities.
- Qualifications.
- Seniority level.
- Domain-specific keywords.
- Soft skills or behavioral signals.

## 7.4 Resume-to-JD Match Scoring

Generate an explainable match score from 0 to 100.

The score should consider:

- Required skill coverage.
- Preferred skill coverage.
- Relevant experience alignment.
- Keyword alignment.
- Responsibility alignment.
- Seniority alignment.
- Missing critical requirements.

The UI or output should include both:

- **Original match score.**
- **Tailored match score.**

The score should be accompanied by a short explanation, not just a number.

## 7.5 Bullet Rewriting

For each relevant resume bullet, the engine should:

- Preserve the user's actual meaning and experience.
- Improve alignment with the JD.
- Use stronger action verbs.

- Include JD-relevant terminology where truthful.
- Preserve or improve measurable impact when present.
- Avoid adding unsupported claims.

Each rewritten bullet should include metadata:

- Original bullet.
- Tailored bullet.
- Reason for change.
- JD keywords addressed.
- Confidence level.
- Risk flag if the rewrite may overstate experience.

## 7.6 Gap Analysis

The system should flag:

- Missing required skills.
- Weakly represented required skills.
- Missing tools or technologies.
- Missing domain experience.
- Missing seniority indicators.
- Unsupported JD requirements that should not be invented.

Each gap should include:

- Gap name.
- Importance: high, medium, or low.
- Evidence from the JD.
- Whether the resume mentions it.
- Suggested action.

Example suggested actions:

- Add if you have this experience.
- Leave out if not true.
- Mention in skills section if familiar.
- Add a project bullet if applicable.
- Prepare to address this in interview.

## 7.7 Truthfulness Guardrails

The product must explicitly avoid fabrication.

The system should not add:

- Employers the user did not work for.
- Degrees or certifications the user does not have.
- Technologies not present in the resume unless marked as a suggested gap.
- Metrics that were not provided.
- Leadership scope that was not implied by the original resume.
- Claims of expert-level proficiency without support.

When uncertain, the system should mark content as a suggestion requiring user confirmation.

## 7.8 Side-by-Side PDF Proof

The MVP should generate a PDF showing:

- Left side: original resume content.
- Right side: tailored resume content.
- Highlighted changed bullets.
- Match score before and after.
- Gap analysis summary.
- JD summary.

This PDF is the main proof artifact for the project.

---

## 8. Suggested User Flow

### Step 1: Upload Resume

User uploads or pastes their resume.

### Step 2: Add Job Description

User pastes a real job listing.

### Step 3: Analyze

System parses both inputs and shows:

- JD summary.
- Extracted requirements.
- Original resume match score.
- Initial gaps.

### Step 4: Generate Tailored Resume

System rewrites relevant bullets and optionally adjusts:

- Resume summary.
- Skills section ordering.
- Project emphasis.
- Experience bullet wording.

### Step 5: Review Changes

User sees side-by-side comparison of original and tailored content.

Each rewritten bullet should have a clear explanation.

## Step 6: Export

User exports:

- Tailored resume PDF.
- Side-by-side comparison PDF.
- Optional markdown or DOCX version.

---

## 9. Functional Requirements

### Resume Parser

The parser should convert uploaded or pasted resume content into structured JSON.

Suggested structure:

```
{
  "contact": {},
  "summary": "",
  "skills": [],
  "experience": [
    {
      "company": "",
      "title": "",
      "startDate": "",
      "endDate": "",
      "bullets": []
    }
  ],
  "projects": [],
  "education": [],
  "certifications": []
}
```

### JD Parser

The parser should convert the job description into structured JSON.

Suggested structure:

```
{
  "jobTitle": "",
  "company": "",
  "requiredSkills": [],
  "preferredSkills": [],
  "responsibilities": [],
  "qualifications": [],
  "tools": [],
}
```

```
"keywords": [],
"seniorityLevel": "",
"domainSignals": []
}
```

## Match Engine

The match engine should output:

```
{
  "overallScore": 0,
  "skillCoverageScore": 0,
  "responsibilityAlignmentScore": 0,
  "keywordScore": 0,
  "seniorityScore": 0,
  "criticalMissingRequirements": [],
  "explanation": ""
}
```

## Tailoring Engine

The tailoring engine should output:

```
{
  "tailoredSummary": "",
  "tailoredSkills": [],
  "tailoredExperience": [
    {
      "company": "",
      "title": "",
      "bullets": [
        {
          "original": "",
          "tailored": "",
          "changeReason": "",
          "keywordsAddressed": [],
          "confidence": "high | medium | low",
          "riskFlag": ""
        }
      ]
    }
  ]
}
```

## Gap Engine

The gap engine should output:



```

{
  "gaps": [
    {
      "name": "",
      "importance": "high | medium | low",
      "jdEvidence": "",
      "resumeEvidence": "",
      "suggestedAction": "",
      "canSafelyAdd": false
    }
  ]
}

```

## PDF Generator

The PDF generator should create two documents:

1. Tailored resume PDF.
2. Side-by-side comparison PDF.

The comparison PDF should include:

- Header with job title and company.
- Original score vs tailored score.
- JD requirements summary.
- Original bullet column.
- Tailored bullet column.
- Highlighted differences.
- Gap analysis section.
- Disclaimer that the user must verify all content before use.

---

## 10. Suggested Architecture

### Frontend

Recommended options:

- Next.js
- React
- Tailwind CSS
- Shadcn UI

Core screens:

- Landing page.
- Resume/JD input page.
- Analysis results page.
- Side-by-side editor.
- Export page.

## Backend

Recommended options:

- Node.js API routes in Next.js.
- Python FastAPI service if document parsing and PDF generation are easier in Python.

Core services:

- Resume parsing service.
- JD parsing service.
- Scoring service.
- LLM tailoring service.
- Gap analysis service.
- PDF generation service.

## Data Storage

For MVP, local/session-based storage is acceptable.

Optional persistent storage:

- PostgreSQL.
- Supabase.
- SQLite for local prototype.

Suggested entities:

- User.
- Resume.
- JobDescription.
- TailoringRun.
- MatchScore.
- GapAnalysis.
- ExportedDocument.

---

## 11. LLM Prompting Requirements

The system should use separate prompts for:

1. JD extraction.
2. Resume parsing cleanup.
3. Resume-JD scoring.
4. Bullet rewriting.
5. Gap analysis.
6. Final resume assembly.

Each LLM output should be requested as strict JSON where possible.

## Important Prompt Rules

The LLM must be instructed to:

- Never invent experience.
  - Use only evidence from the resume.
  - Mark uncertain suggestions clearly.
  - Keep bullet length resume-appropriate.
  - Prefer concrete impact and measurable outcomes.
  - Avoid keyword stuffing.
  - Preserve the user's original career level.
  - Explain every meaningful rewrite.
- 

## 12. Example Acceptance Criteria

The project is complete when a user can:

1. Paste a resume.
  2. Paste a real job description.
  3. Click an analyze button.
  4. See the original resume match score.
  5. See extracted JD requirements.
  6. See missing or weakly represented requirements.
  7. Generate a tailored resume.
  8. Review original vs tailored bullets side by side.
  9. See a tailored match score.
  10. Export a side-by-side PDF showing original vs tailored resume.
- 

## 13. Demo Requirement

The final demo should use a real job listing and a real or realistic sample resume.

The demo should produce:

- Original resume text.
- Job description text.
- JD analysis.
- Original match score.
- Tailored resume.
- Tailored match score.
- Gap analysis.
- Side-by-side PDF comparison.

The proof artifact should clearly show that Resume Shapeshifter improves resume alignment without fabricating experience.

---

## 14. Quality Bar

The generated tailored resume should be:

- Truthful.
- Concise.
- ATS-friendly.
- Specific to the job description.
- Easy to review.
- Exportable.
- Clearly different from the original where relevant.
- Not overloaded with unnatural keywords.

The gap analysis should be actionable and honest.

The scoring should be explainable rather than opaque.

---

## 15. Suggested Initial Implementation Plan

### Phase 1: Static Prototype

- Build UI for resume and JD input.
- Use pasted text only.
- Mock parsing and scoring.
- Render side-by-side comparison in browser.

### Phase 2: LLM Integration

- Add JD extraction prompt.
- Add resume parsing prompt.
- Add scoring prompt.
- Add bullet rewrite prompt.
- Add gap analysis prompt.

### Phase 3: PDF Export

- Generate tailored resume PDF.
- Generate comparison PDF.
- Add highlighted changes.

### Phase 4: Validation and Guardrails

- Add unsupported-claim detection.
- Add confidence labels.
- Add user confirmation flags.
- Add stricter JSON schema validation.

### Phase 5: Polish

- Improve UI.
- Add sample resume and JD.
- Add download buttons.

- Add loading states and error handling.

---

## 16. Recommended Tech Stack

A practical Cursor-friendly stack:

- **Frontend:** Next.js, React, TypeScript, Tailwind CSS, Shadcn UI
- **Backend:** Next.js API routes or FastAPI
- **LLM:** OpenAI API or another structured-output-capable LLM
- **Validation:** Zod
- **PDF Generation:** Playwright PDF, React PDF, or Puppeteer
- **Document Parsing:** pdf-parse, mammoth, or Python alternatives
- **Storage:** SQLite, Supabase, or local JSON for MVP

---

## 17. Risks and Edge Cases

### Parsing Risks

- Multi-column resumes may parse poorly.
- PDF formatting may produce broken text order.
- Resume sections may have non-standard names.

### LLM Risks

- The model may overstate experience.
- The model may add unsupported keywords.
- The model may produce inconsistent JSON.
- The score may appear more precise than it really is.

### Product Risks

- Users may trust generated content without reviewing it.
- Users may expect guaranteed ATS performance.
- Job descriptions may be vague or overly broad.

### Mitigations

- Add strong truthfulness instructions.
- Validate JSON responses.
- Show evidence for each rewrite.
- Include disclaimers.
- Require user review before export.
- Flag low-confidence changes.

---

## 18. Cursor Development Instructions

When implementing this project, prioritize a working vertical slice over broad feature coverage.

Start with:

1. A single-page app where the user pastes resume text and JD text.
2. A server route that calls the LLM and returns structured JSON.
3. A scoring section.
4. A rewritten bullets section.
5. A gap analysis section.
6. A side-by-side preview.
7. A PDF export button.

Use TypeScript types or Zod schemas for every major object:

- ResumeProfile
- JobDescriptionProfile
- MatchScore
- TailoredResume
- ResumeGap
- TailoringRun

Keep prompts in separate files, for example:

```
/prompts/jd-extraction.ts
/prompts/resume-parser.ts
/prompts/match-scoring.ts
/prompts/bullet-rewriter.ts
/prompts/gap-analysis.ts
```

Keep rendering components separate from business logic:

```
/components/ResumeInput.tsx
/components/JDInput.tsx
/components/ScoreCard.tsx
/components/GapAnalysis.tsx
/components/SideBySideDiff.tsx
/components/PDFExportButton.tsx
/lib/scoring.ts
/lib/prompts.ts
/lib/pdf.ts
/lib/schemas.ts
```

---

## 19. Definition of Done

The project is done when the app can generate a complete side-by-side PDF for a real job listing, including:

- Original resume.
- Tailored resume.
- Original match score.

- Tailored match score.
- JD keyword and requirement summary.
- Bullet-level rewrite explanations.
- Gap analysis.
- Truthfulness disclaimer.

The output should be polished enough to share as a portfolio project or demo.

---

## 20. One-Line Product Description

Resume Shapeshifter turns any job description into a truthful, targeted resume rewrite with match scoring, gap analysis, and a side-by-side PDF proof artifact.